

MIMIC-IV Note 기반 RAG 데이터 구축 및 검색 파이프라인 보고서

0. 핵심 설계방향 설명(최종 보고서 작성 시에 참고할 부분입니다)

MIMIC-IV Note 기반 의료 원문을 단순 임베딩하지 않고, 임상적 의미와 시간적 맥락을 유지한 검색 지식베이스로 변환하는 것을 목적으로 함

- 의사 소견, 퇴원 요약지, 영상의학 판독문을 환자·입원 단위로 연결하여 임상 맥락 보존
- Radiology 원문을 섹션 단위로 분리하고, Bedrock 기반 의미 추출을 통해 양성/음성 소견 구조화
- LLM 추출 결과를 규칙 기반 QC와 LLM 보강 단계로 검증하여 원문 대비 누락과 오분류 위험 축소
- 정제된 discharge_summary와 radiology 청크를 Titan Embed v2로 임베딩하고 ChromaDB에 적재
- RAG 모듈을 멀티모달 분석 결과의 임상적 해석과 최종 소견 근거 보강 계층으로 활용

1. 아키텍처 개요

데이터 처리 계층에서는 MIMIC-IV Note 원문을 그대로 벡터화하지 않고, 구조적 섹서닝, Bedrock 기반 의미 추출, 규칙 기반 QC, LLM 보강, 정제 청킹, 퇴원 요약지와 병합, 임베딩 및 벡터 DB 적재 순서로 처리함. 이 구조의 목적은 의료 문서에 포함된 검사 종류, 이상 소견, 정상 또는 부재 소견, 시간적 변화, 환자 단위 식별 정보를 검색 가능한 형태로 보존하는 것임.

검색 계층에서는 Titan Embed v2를 사용하여 query embedding을 생성하고, ChromaDB PersistentClient 기반 local_rag_db에서 Top-K 유사 사례를 검색함. 검색 결과는 단순 유사도 상위 결과만 사용하는 것이 아니라, discharge_summary와 radiology가 함께 포함되도록 다양성 필터링을 적용함.

생성 계층에서는 검색된 과거 유사 사례와 현재 환자의 입력 또는 모달 분석 결과를 함께 사용하여 Claude 기반 최종 응답을 생성함. RAG 모듈은 예측 모델을 대체하는 구조가 아니라, ECG, CXR, LAB 등 멀티모달 결과가 의미하는 임상적 맥락을 과거 유사 사례와 연결하여 설명 가능성과 근거성을 높이는 보조 실행 계층으로 정의함.

2. 전체 데이터 처리 파이프라인

전체 파이프라인은 원문 데이터의 구조화에서 시작하여 최종 RAG 응답 생성까지 이어짐. 특히 LLM 결과를 그대로 사용하지 않고, QC와 보강 단계를 분리하여 추적 가능한 방식으로 품질을 개선한 점이 핵심임.

- Step 1. Radiology 원문 섹서닝
- Step 2. Bedrock 기반 시계열 의미 추출
- Step 3. 보강 병합 및 고정밀 청킹
- Step 4. 퇴원 요약지와 영상의학 기록 병합
- Step 5. Titan Embed v2 임베딩 및 ChromaDB 적재
- Step 6. RAG 오케스트레이션 및 최종 응답 생성

이 파이프라인은 의료 원문을 검색 가능한 임상 근거로 변환하는 과정임. 원문을 바로 벡터화하지 않고 섹션, 양성/음성 소견, 시간 변화, 환자 단위 메타데이터를 보존하도록 단계가 설계됨.

3. 데이터 소스 및 최종 산출물 요약

데이터 처리 대상은 타겟 hadm_id 기준의 MIMIC-IV Note 관련 문서임. 영상의학 판독문과 퇴원 요약지를 병합하여 최종 지식베이스를 구성하였고, local_rag_db의 README 및 SQLite 메타데이터 기준 최종 ChromaDB 컬렉션은 다음과 같이 정리됨.

항목	내용	설계 의미
최종 벡터 DB	local_rag_db / ChromaDB PersistentClient	로컬 환경에서도 재현 가능한 검색 기반 구성
컬렉션명	medical_rag_collection	RAG 검색 대상 컬렉션 단일화
총 문서 수	49,743건	퇴원 요약지와 영상의학 청크를 통합 저장
퇴원 요약지 청크	9,998건	입원 단위의 전반적 임상 맥락 제공
영상의학 청크	39,745건	검사별 세부 영상 소견 제공
고유 hadm_id 커버리지	9,999 / 10,000	타겟 입원 단위 대부분을 검색 대상으로 포함
임베딩 모델	Amazon Titan Embed Text v2	AWS Bedrock 기반 임베딩 체계 적용
임베딩 차원	512차원	검색 품질과 저장 효율을 함께 고려
유사도 메트릭	cosine	의미 기반 텍스트 검색에 적합한 거리 기준 사용
DB 용량	약 372MB	컨테이너 시작 시 다운로드 가능한 규모로 관리
QC 판정	PASS	기본 무결성 및 검색 품질 검증 통과

4. 단계별 스크립트 구성 및 역할

본 장은 scripts.zip에 포함된 주요 스크립트를 실제 실행 흐름 기준으로 정리한 것임. Swarm 계열 스크립트는 벡터 DB 선택, 서비스 DB 구조 검토, 문서화 보조 목적이므로 주요 RAG 구축 파이프라인과 분리함.

단계	파일명	주요 목적
Step 1	step1_sectioning.py	radiology.csv에서 타겟 hadm_id만 필터링하고, 딕셔너리 기반 정규식으로 EXAM_TECH, FINDINGS, IMPRESSION 섹션을 분리함
Step 2	step2_bedrock_processor.py	환자별 영상의학 보고서를 시간순으로 처리하며 Claude Haiku를 이용해 modality, clinical_status, positive/negative findings를 JSON으로 추출함
Step 3	step3_chunking.py	LLM 보강 결과를 병합하고 normalcy filter, fuzzy dedup을 적용한 뒤 embedding_text와 metadata를 생성함
Step 4	step4_merger.py	퇴원 요약지와 영상의학 청크를 subject_id 기준으로 병합하고, radiology_charttime <= discharge_charttime 조건을 적용함
Step 5	step5_local_embedding_ingestion.py	퇴원 요약지와 영상의학 기록을 평탄화하고 Titan Embed v2 512차원 임베딩을 생성해 ChromaDB에 적재함
Step 6	step6_rag_orchestrator.py	사용자 입력을 Titan 임베딩으로 변환하고 ChromaDB에서 검색한 뒤 Claude 기반 최종 응답을 생성함

5. 단계별 핵심 설계 의도와 중요 고려사항

본 파트에서 중요한 것은 각 스크립트가 무엇을 실행했는지보다, 왜 해당 방식으로 설계되었는지임. 의료 RAG에서는 원문 의미 보존, 시간 순서, 양성/음성 소견 분리, 보강 결과의 추적 가능성이 검색 품질과 안전성에 직접적으로 영향을 줌.

5.1 Step 1: 구조적 섹셔닝

영상의학 보고서는 일반 자연어 문서처럼 보이지만, 실제로는 EXAMINATION/TECHNIQUE, FINDINGS, IMPRESSION 등 섹션별 임상 의미가 다름. 따라서 원문 전체를 한 번에 임베딩하면 검사 기법, 정상 소견, 실제 이상 소견이 섞여 검색 품질이 낮아질 수 있음.

- 타겟 hadm_id만 필터링하여 프로젝트 범위에 맞는 데이터만 처리
- 섹션 디셔너리의 유의어를 longest match first 방식의 정규식으로 변환
- FINDINGS와 ORGAN_SPECIFIC_FINDINGS는 raw_findings로 라우팅
- IMPRESSION은 raw_impression으로, EXAMINATION/TECHNIQUE는 raw_exam_tech로 라우팅
- COMPARISON, INDICATION, ADMINISTRATIVE 등 검색 핵심도가 낮은 섹션은 후속 구조화 대상에서 제외

이 단계의 목적은 텍스트 길이를 줄이는 것이 아니라, 의료 문서 안에서 임상적 의미가 다른 부분을 구분하는 것임. 섹셔닝이 안정적으로 수행되어야 이후 LLM 추출과 청킹 결과도 일관되게 유지됨.

5.2 Step 2: Bedrock 기반 시계열 의미 추출

Step 2는 Bedrock Claude Haiku를 이용해 영상의학 보고서를 구조화된 JSON으로 변환하는 단계임. 환자별 보고서를 시간순으로 처리하고 이전 검사 결과를 Prior Context로 넘겨 baseline, stable, worsening, improving, resolved 같은 시간적 상태를 반영하도록 설계함.

- modality를 CXR, CT, MRI, US, XR, PET, Other 중 하나로 분류
- positive_findings에는 실제 존재하는 이상 소견만 포함하도록 프롬프트에서 제한
- negative_findings에는 명시적으로 부재, 정상, 배제된 소견만 포함
- 수치와 단위가 있는 병변 정보는 삭제하지 않고 보존
- 첫 검사에서 baseline이 아닌 상태가 나오면 코드에서 baseline으로 강제 보정
- 체크포인트, 에러 로그, 재시도 로직으로 대량 처리 중 중단 및 재실행 가능성 확보

의료 RAG에서 가장 위험한 오류 중 하나는 없음 또는 정상으로 기록된 소견이 실제 존재하는 소견처럼 검색되는 것임. 따라서 양성 소견과 음성 소견을 분리하고, negation leak을 방지하는 프롬프트와 후처리 규칙이 중요함.

5.3 Step 3: 보강 병합 및 고정밀 청킹

Step 3은 실제 영상의학 검색 청크를 만드는 단계임. LLM 보강 결과를 병합하고, 정상 소견이 positive_findings에 남아 있으면 negative 쪽으로 이동시키며, positive와 negative 간 유사 중복을 제거함. 이후 환자별 시계열 순서를 유지한 상태로 embedding_text와 metadata를 생성함.

- qc_supplement의 added_positive와 added_negative를 기존 결과에 병합

- negation_corrections는 positive에서 제거하고 negative로 이동
- normalcy filter로 정상 또는 부재 성격의 표현을 positive에서 제거
- fuzzy dedup으로 positive와 negative에 유사 항목이 동시에 남는 문제 방지
- embedding_text는 Exam Type, Clinical Status, Positive Findings, Explicitly Ruled Out 구조로 통일
- metadata에는 note_id, subject_id, hadm_id, charttime, modality, clinical_status, event_sequence, total_exams를 포함

청킹의 핵심은 문서를 단순히 잘게 나누는 것이 아니라, 검색 쿼리와 잘 매칭될 수 있도록 임상적으로 의미 있는 단위와 문장 구조를 만드는 것임. 특히 positive/negative 분리는 검색 결과의 안전성과 직결됨.

5.4 Step 4: 퇴원 요약지와 영상의학 기록 병합

Step 4는 radiology 청크와 discharge summary를 통합하는 단계임. 같은 subject_id의 영상의학 기록을 퇴원 요약지에 연결하되, 영상의학 검사 시간이 퇴원 요약지 시간보다 이전이거나 같은 경우만 포함하도록 temporal join을 적용함.

- note_id 기준으로 radiology 중복 제거
- subject_id 기준으로 환자별 영상의학 기록 그룹화
- radiology_charttime <= discharge_charttime 조건으로 미래 정보가 섞이는 문제 방지
- 퇴원 요약지 레코드에 radiology_history 필드를 추가하여 이기종 의료 문서를 하나의 입원 단위 지식으로 통합

의료 RAG에서는 시간 순서가 중요함. 미래 검사 결과가 과거 진단 근거처럼 들어가면 데이터 누수와 잘못된 임상 해석이 발생할 수 있으므로, 시간 조건을 둔 병합이 필요함.

5.5 Step 5: 임베딩 생성 및 ChromaDB 적재

Step 5는 통합 지식 레코드를 실제 검색 가능한 벡터 DB로 변환하는 단계임. 퇴원 요약지는 ml_features 값을 재귀적으로 평탄화하고, 영상의학 기록은 Step 3에서 생성한 embedding_text를 사용함. 이후 Titan Embed v2로 512차원 임베딩을 생성하여 ChromaDB에 저장함.

- discharge_summary 청크 ID는 {hadm_id}_discharge 형식으로 생성
- radiology 청크 ID는 {hadm_id}_rad_{index} 형식으로 생성
- 8,000자 truncation으로 모델 입력 제한 방어
- BATCH_SIZE 100 단위로 ChromaDB에 upsert
- hadm_id 기준 체크포인트로 중단 후 재실행 가능
- BaseVectorDB 추상 인터페이스를 두어 향후 OpenSearch 등 다른 벡터 DB로 전환 가능하도록 설계

로컬 ChromaDB 선택은 시연 및 학교 프로젝트 환경에서 비용과 재현성을 확보하기 위한 선택임. 동시에 BaseVectorDB 구조를 둔 것은 추후 운영 환경 전환 가능성을 열어둔 설계로 볼 수 있음.

5.6 Step 6: RAG 오케스트레이션

Step 6은 사용자 입력 또는 모달 결과를 검색 쿼리로 사용하여 최종 응답을 생성하는 단계임. 입력 텍스트를 Titan Embed v2로 임베딩하고, ChromaDB에서 Top-20 후보를 검색한 뒤 disc

harge_summary와 radiology가 모두 포함되도록 다양성 필터링을 수행함. 최종 Top-3를 Claude 프롬프트에 넣어 근거 기반 응답을 생성함.

- CHROMA_MODE 설정으로 local/remote ChromaDB 선택 가능
- 임베딩 캐시를 통해 동일 쿼리 반복 호출 비용과 지연시간 감소
- 최고 유사도가 기준 미달이면 fallback 응답 반환
- Top-20 검색 후 discharge_summary와 radiology를 최소 1건씩 포함하도록 다양성 필터링
- 시스템 프롬프트에서 현재 환자의 정상/비정상 분리, 과거 사례의 제한적 활용, 구체적인 수치/위치 명시를 강제

6. 임베딩 및 ChromaDB 구축 결과

최종 로컬 벡터 DB는 49,743개의 문서 청크를 포함함. 구조적으로는 퇴원 요약지와 영상의학 청크를 동일 컬렉션에 저장하되, chunk_type 메타데이터로 문서 유형을 구분함. 검색 시 이 메타데이터를 활용하면 유형별 검색 결과 필터링이나 다양성 확보가 가능함.

구분	ID 형식 및 건수	주요 메타데이터
discharge_summary	{hadm_id}_discharge / 9,998건	chunk_type, subject_id, hadm_id
radiology	{hadm_id}_rad_{index} / 39,745건	chunk_type, subject_id, hadm_id, note_id, charttime, modality, clinical_status, event_sequence, total_exams

이 구성은 한 환자 또는 입원 단위의 전반적 임상 맥락은 discharge_summary에서 가져오고, 검사별 세부 영상 소견은 radiology에서 가져오도록 설계한 것임. Step 6의 다양성 필터링은 이 구조를 활용하여 최종 컨텍스트가 한 종류의 문서로만 편향되는 것을 방지함.

7. RAG 검색 및 응답 생성 구조

최종 RAG 실행 구조는 Retrieval, Augmentation, Generation으로 구분할 수 있음. 검색 결과는 단순히 많이 가져오는 것이 아니라, 최종 응답에 사용할 수 있는 근거 품질을 유지하도록 후보 확보, 다양성 필터링, fallback 판단, 생성 프롬프트 제어 순서로 처리함.

구성 요소	처리 내용	설계 의도
Retrieval	사용자 입력 또는 모달 결과를 Titan Embed v2로 임베딩하고 ChromaDB에서 Top-20 후보 검색	충분한 후보군을 먼저 확보하여 검색 누락 위험 축소
Diversity Filtering	discharge_summary와 radiology를 최소 1건씩 포함해 Top-3 구성	퇴원 요약지 중심의 전체 맥락과 영상의학 중심의 검사 근거를 함께 반영
Fallback	최고 유사도가 MIN_SIMILARITY 미만이면 유사 사례 없음 응답 반환	낮은 근거 품질에서 과도한 추론이 발생하는 것을 방지
Generation	Claude 모델에 현재 환자 검사 결과와 과거 유사 사례를 함께 제공해 5개 항목으로 소견 생성	근거 기반 통합 소견 작성

응답 생성 프롬프트는 단순 요약이 아니라 의료적 안전성을 고려한 구조로 작성됨. 특히 데이터 없음과 정상 소견을 구분하고, 과거 유사 사례는 현재 환자의 비정상 항목을 해석할 때만 참고하도록 제한한 점이 중요함.

8. 멀티모달 시스템 내 RAG 모듈의 역할

RAG 모듈은 ECG, CXR, LAB 모듈이 산출한 예측값 또는 소견을 반복하는 역할이 아님. 각 모듈 결과가 의미하는 임상적 맥락을 과거 유사 환자 사례와 연결하여 최종 통합 판단의 근거를 보강하는 역할을 수행함.

입력 또는 연계 대상	RAG에서의 활용 방식	시스템 관점의 의미
CXR 모듈 결과	findings_text와 impression을 검색 쿼리로 변환하여 유사 영상의학 사례 검색	현재 영상 소견과 과거 유사 영상 소견을 연결
ECG 모듈 결과	findings, ecg_vitals, 고확률 예측 질환명을 자연어 쿼리로 변환하여 관련 임상 사례 검색	시계열 신호 분석 결과에 임상 사례 맥락 부여
LAB 결과	감염, 전해질 이상, 신장 기능, 대사 이상 등 비정상 수치 해석 시 과거 유사 사례 검색	수치형 검사 결과의 임상적 해석 보조
중앙 오케스트레이터	각 모듈 결과를 종합한 뒤 RAG 검색 결과를 최종 임상 소견의 근거로 활용	모달 분석과 근거 검색을 연결하는 통합 흐름 구성
최종 사용자	현재 환자 검사 결과를 직접 입력하여 과거 유사 사례 기반 설명과 권고사항 확인	설명 가능성과 판단 근거 제공

따라서 RAG는 예측 모델을 대체하지 않고, 예측 결과의 임상적 해석과 설명 가능성을 보강하는 보조 모듈로 기능함.

9. 품질 검증 방식

품질 검증은 주요 작업 흐름을 보조하는 수준으로 설계함. 본 보고서에서는 파이프라인 구성과 설계 의도를 중심으로 설명하고, 검증 항목은 핵심만 정리함.

검증 구분	주요 내용	의미
규칙 기반 QC	temporal mismatch, empty findings, pos/neg overlap, high compression, negation leak 탐지	명확한 구조화 오류를 자동으로 탐지
LLM 보장	QC flagged 건과 통과 건 일부 샘플에 대해 원문 대비 누락/오분류 확인	자동 추출 결과의 누락 위험 보완
임베딩 평가	기본 통계, 벡터 품질, 의미 검색, 환자 내 유사도, 문서 유형 간 분리 확인	검색 인덱스의 기본 품질 확인
ChromaDB QC	총 문서 수, hadm_id 커버리지, 벡터 차원, 검색 평균 유사도 등을 확인하고 PASS/FAIL 판정	최종 DB 무결성 검증
모달 연계 평가	CXR/ECG 모듈 결과를 쿼리로 사용해 Hit@K, MR R 확인	멀티모달 입력과 RAG 검색 연결성 점검

10. 결론

본 RAG 구축 작업의 핵심 성과는 MIMIC-IV Note 기반 의료 원문을 단순 임베딩 대상이 아니라, 임상적 의미와 시간적 맥락을 갖춘 검색 지식베이스로 변환했다는 점임. Step 1의 섹셔닝, Ste

p 2의 Bedrock 의미 추출, Step 3의 정제 청킹, Step 4의 시간 조건 병합, Step 5의 임베딩 및 적재, Step 6의 검색/생성 오케스트레이션이 하나의 연속된 파이프라인으로 구성됨.

특히 중요하게 고려한 부분은 양성 소견과 음성 소견의 분리, 시간적 정보 보존, 원본과 보강 결과의 추적 가능성, discharge_summary와 radiology의 균형 있는 검색임. 이 구조는 멀티모달 AI 시스템에서 RAG 모듈이 단순 참고 문서 검색을 넘어, 예측 결과의 임상적 해석과 설명 가능성을 보장하는 근거 기반 모듈로 사용될 수 있음을 보여줌.

향후 운영 단계에서는 Human Review Gold Set, 정량 검색 평가, Hybrid Search, Reranker, 프롬프트 및 데이터 버전 관리, 운영형 Vector DB 전환 등을 통해 검색 정확도와 운영 안정성을 추가로 강화할 수 있음.